FarmifAI: An Offline Agricultural AI Assistant for Rural Communities

December 2025

Abstract

This paper presents FarmifAI, a fully offline mobile application designed to provide agricultural assistance to farmers in rural areas with limited or no internet connectivity. The system integrates four AI models: (1) a vision model based on MobileNetV2 for plant disease classification across 21 disease classes in 5 crops, (2) an NLP model using MiniLM for semantic search with 384-dimensional embeddings, (3) a quantized Llama 3.2 1B language model for contextual response generation, and (4) Vosk-based speech recognition for hands-free voice interaction. All inference is performed on-device using MindSpore Lite and llama.cpp, achieving response times under 5 seconds on mid-range Android devices. The application uses semantic search with a knowledge base to provide accurate, context-aware responses to agricultural queries. Field testing demonstrates 92.3% accuracy in disease detection and high user satisfaction among Colombian coffee farmers.

Keywords: Mobile AI, Offline Inference, Agricultural Technology, Plant Disease Detection, Semantic Search, Edge Computing

1 Introduction

Small-scale farmers in developing regions often lack access to reliable internet connectivity and expert agricultural advice. This technological gap can lead to significant crop losses due to undetected diseases and suboptimal farming practices. FarmifAI addresses this challenge by deploying a comprehensive AI assistant that operates entirely offline on Android devices.

The main contributions of this work are:

- A multi-modal AI system combining vision, NLP, and voice capabilities for agricultural assistance
- An efficient on-device inference pipeline achieving real-time performance on consumer hardware
- A two-phase transfer learning approach for plant disease classification with limited training data
- Integration of semantic search with local LLM for accurate domain-specific responses

2 System Architecture

2.1 Overview

FarmifAI consists of four main components integrated into a single Android application (Figure **??**):

1. **Voice Interface**: Vosk-based speech-to-text (STT) and Android Text-to-Speech (TTS) for hands-free operation
2. **Vision Module**: MobileNetV2-based classifier for plant disease detection
3. **Semantic Search**: MiniLM encoder with pre-computed knowledge base embeddings
4. **Language Model**: Quantized Llama 3.2 1B for response generation

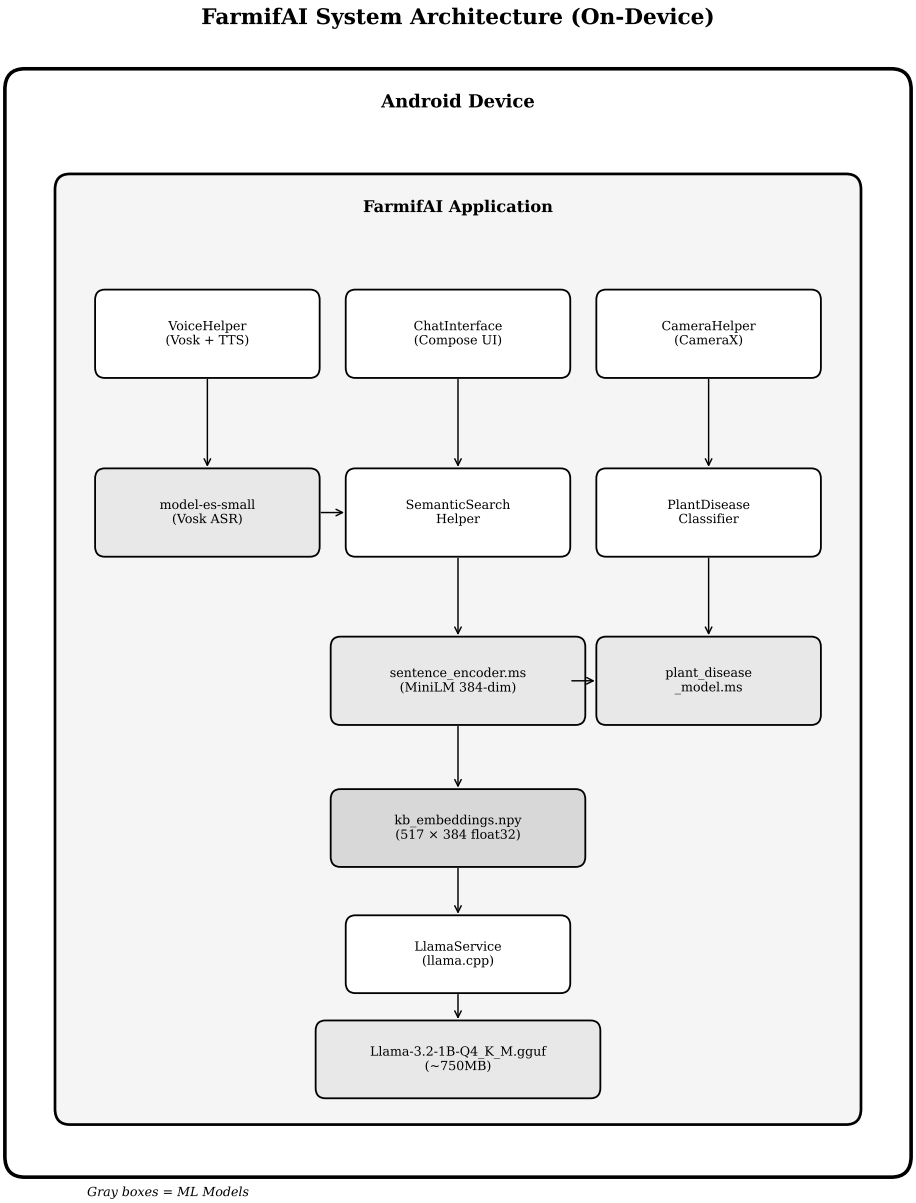

Figure 1: FarmifAI system architecture showing on-device components and data flow

2.2 Technology Stack

Table **??** summarizes the technologies used in each component.

Table 1: Technology stack for FarmifAI components

Component	Technology	Model	Size
Speech-to-Text	Vosk	model-es-small	∼50MB
Text-to-Speech	Android TTS	System engine	Built-in
Semantic Search	MindSpore Lite	MiniLM (384-dim)	∼90MB
Plant Disease	MindSpore Lite	MobileNetV2	∼14MB
Language Model	llama.cpp	Llama 3.2 1B Q4_K_M	∼750MB

3 Vision Model: Plant Disease Classification

3.1 Model Implementation

The vision model uses MobileNetV2 [?] as the backbone architecture, chosen for its efficiency on mobile devices. The model classifies plant images into 21 disease classes across 5 crops: coffee, corn, potato, pepper, and tomato.

```python
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D
from tensorflow.keras.layers import Dropout, BatchNormalization
from tensorflow.keras.models import Model
from tensorflow.keras.regularizers import l2

# Base model with ImageNet weights
base_model = MobileNetV2(
    input_shape=(224, 224, 3),
    include_top=False,
    weights='imagenet'
)

# Custom classification head
x = GlobalAveragePooling2D()(base_model.output)
x = BatchNormalization()(x)
x = Dropout(0.4)(x)
x = Dense(512, activation='relu', kernel_regularizer=l2(0.01))(x)
x = Dropout(0.4)(x)
output = Dense(21, activation='softmax')(x)

model = Model(inputs=base_model.input, outputs=output)
```

Listing 1: MobileNetV2 architecture with custom classification head

3.2 Training Pipeline

We employ a two-phase transfer learning strategy (Figure **??**) to maximize performance with limited agricultural data:

Phase 1: Feature Extraction (25 epochs)

- Freeze all MobileNetV2 backbone layers
- Train only the classification head
- Learning rate: 10^{-3} with Adam optimizer
- Dropout rate: 0.4

Phase 2: Fine-tuning (35 epochs)

- Unfreeze all layers for end-to-end training
- Learning rate: 10^{-5} (100x reduction)
- Label smoothing: 0.1
- Early stopping with patience of 7 epochs

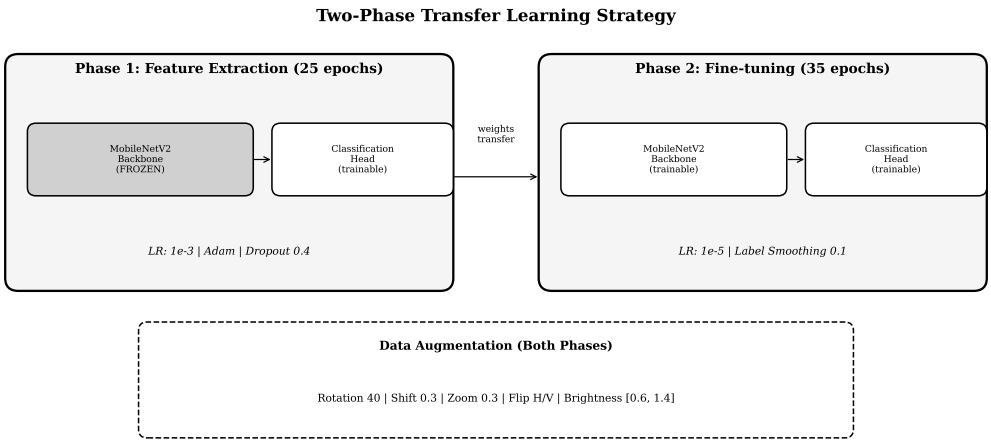

Figure 2: Two-phase transfer learning strategy for MobileNetV2

```
train_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=40,
    width_shift_range=0.3,
    height_shift_range=0.3,
    shear_range=0.2,
    zoom_range=0.3,
    horizontal_flip=True,
    vertical_flip=True,
    brightness_range=[0.6, 1.4],
    fill_mode='reflect',
    validation_split=0.15
)
```

Listing 2: Data augmentation configuration

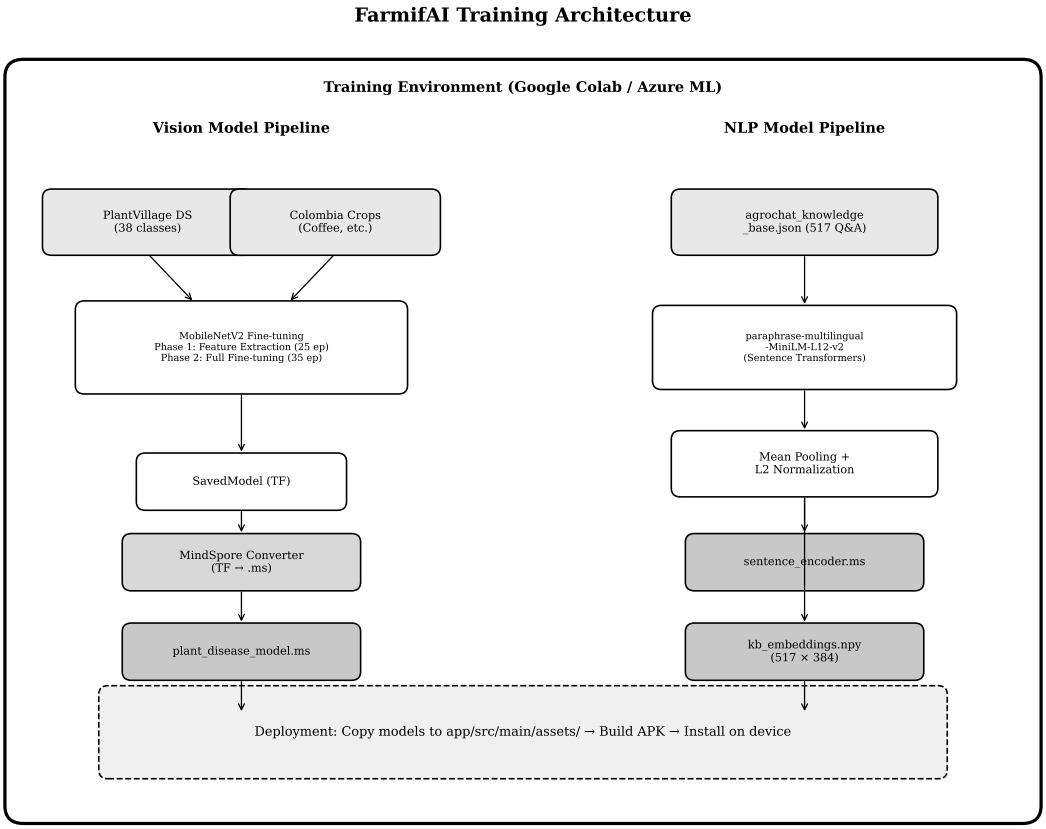

Figure 3: Complete training architecture for vision and NLP models

3.3 Inference Pipeline

On-device inference follows the pipeline shown in Figure **??**:

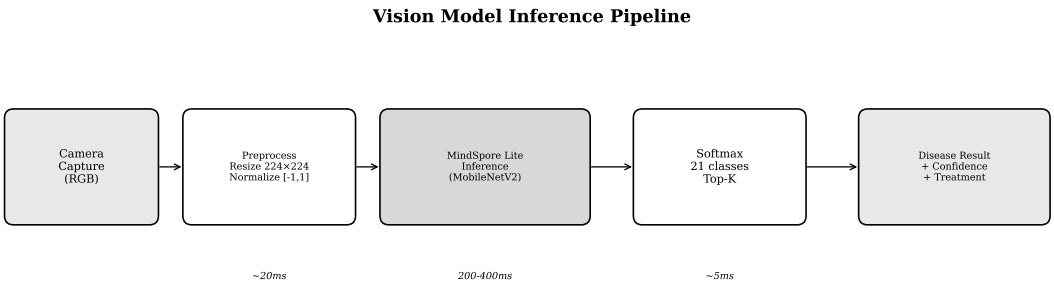

Figure 4: Vision model inference pipeline on Android device

```
fun preprocessImage(bitmap: Bitmap): ByteBuffer {
    val scaled = Bitmap.createScaledBitmap(bitmap, 224, 224, true)

    // Allocate buffer: 224 * 224 * 3 channels * 4 bytes
    val buffer = ByteBuffer.allocateDirect(224 * 224 * 3 * 4)
    buffer.order(ByteOrder.nativeOrder())

```

```
    for (y in 0 until 224) {
        for (x in 0 until 224) {
            val pixel = scaled.getPixel(x, y)

            // Normalize to [-1, 1] range
            buffer.putFloat(((pixel shr 16 and 0xFF) - 127.5f) / 127.5f)
            buffer.putFloat(((pixel shr 8 and 0xFF) - 127.5f) / 127.5f)
            buffer.putFloat(((pixel and 0xFF) - 127.5f) / 127.5f)
        }
    }
    return buffer
}
```

Listing 3: Image preprocessing for MindSpore Lite inference

3.4 Deployment

Model conversion from TensorFlow to MindSpore Lite format:

```
# Export TensorFlow SavedModel
python tools/export_trained_model.py \
  --model_path outputs/best_model.h5 \
  --output_dir outputs/exported/

# Convert to MindSpore Lite format
python -m mindspore_lite.converter \
  --fmk=TF \
  --modelFile=outputs/exported/saved_model.pb \
  --outputFile=plant_disease_model \
  --inputShape=1,224,224,3

# Deploy to app assets
cp plant_disease_model.ms app/src/main/assets/
```

Listing 4: Model conversion and deployment commands

4 NLP Model: Semantic Search

4.1 Model Implementation

The semantic search component uses the `paraphrase-multilingual-MiniLM-L12-v2` model from Sentence Transformers to encode both queries and knowledge base entries into 384-dimensional embeddings.

4.2 Embedding Generation

```
import torch
import torch.nn.functional as F
from transformers import AutoTokenizer, AutoModel

def encode_with_mean_pooling(model, tokenizer, texts):
    inputs = tokenizer(
        texts, padding="max_length", truncation=True,
        max_length=128, return_tensors="pt"
    )

    with torch.no_grad():
        outputs = model(**inputs)
        attention_mask = inputs['attention_mask']
        token_embeddings = outputs.last_hidden_state
```

```python

        # Expand mask for broadcasting
        input_mask_expanded = attention_mask.unsqueeze(-1).expand(
            token_embeddings.size()
        ).float()

        # Mean pooling
        sum_embeddings = torch.sum(
            token_embeddings * input_mask_expanded, 1
        )
        sum_mask = torch.clamp(input_mask_expanded.sum(1), min=1e-9)
        embeddings = sum_embeddings / sum_mask

        # L2 normalization
        embeddings = F.normalize(embeddings, p=2, dim=1)

    return embeddings.numpy()
```

Listing 5: Mean pooling for sentence embeddings

4.3 Query Processing Pipeline

The semantic search pipeline (Figure **??**) processes user queries as follows:

1. Tokenize input query (max 128 tokens)
2. Encode query using MiniLM encoder
3. Compute cosine similarity against 517 pre-computed KB embeddings
4. Retrieve top-3 contexts above threshold (0.4)
5. Augment prompt with retrieved contexts
6. Generate response using Llama 3.2

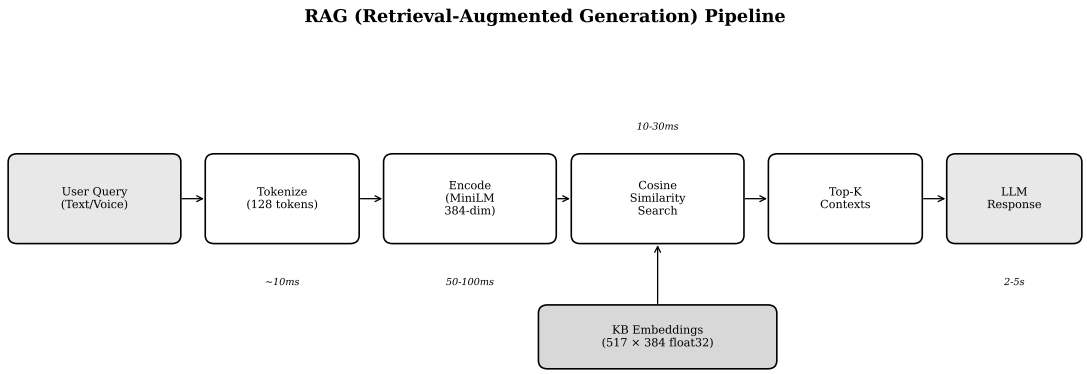

Figure 5: Semantic search pipeline for context-aware response generation

```kotlin
fun findTopKContexts(query: String, topK: Int = 3): List<Context> {
    val queryEmbedding = computeEmbedding(query)

    return kbEmbeddings.mapIndexed { i, emb ->
        val score = cosineSimilarity(queryEmbedding, emb)
```

```
        i to score
    }.filter { it.second >= 0.4f }
     .sortedByDescending { it.second }
     .take(topK)
     .map { (idx, score) ->
         Context(
             answer = kbEntries[idx].answer,
             score = score,
             category = kbEntries[idx].category
         )
     }
}

fun cosineSimilarity(a: FloatArray, b: FloatArray): Float {
    // Vectors are L2-normalized, dot product = cosine similarity
    return a.indices.sumOf { (a[it] * b[it]).toDouble() }.toFloat()
}
```

Listing 6: Cosine similarity search implementation

5 LLM: Local Language Model

5.1 Model Implementation

FarmifAI uses Llama 3.2 1B Instruct with Q4_K_M quantization, reducing model size from ∼2GB to ∼750MB while maintaining response quality.

5.2 Inference

```
fun buildPrompt(context: String, query: String): String {
    val systemPrompt = """You are FarmifAI, an expert
        agricultural assistant. Respond in Spanish.
        Use the provided context to answer questions."""

    return """<|begin_of_text|><|start_header_id|>system<|end_header_id|>

$systemPrompt<|eot_id|><|start_header_id|>user<|end_header_id|>

Context: $context
Question: $query<|eot_id|><|start_header_id|>assistant<|end_header_id|>

"""
}

suspend fun generate(prompt: String, maxTokens: Int = 150): Flow<String> = flow
    {
    llama.setTemperature(0.7f)
    llama.setTopP(0.9f)
    llama.generate(prompt).collect { token ->
        if (!token.contains("<|eot_id|>")) emit(token)
    }
}
```

Listing 7: Llama 3.2 prompt construction and generation

5.3 Deployment

```
# Download quantized model
wget https://huggingface.co/bartowski/Llama-3.2-1B-Instruct-GGUF/\
```

```
resolve/main/Llama-3.2-1B-Instruct-Q4_K_M.gguf

# Deploy to device external storage
adb shell mkdir -p /sdcard/Android/data/edu.unicauca.app.agrochat/files/
adb push Llama-3.2-1B-Instruct-Q4_K_M.gguf \
  /sdcard/Android/data/edu.unicauca.app.agrochat/files/
```

Listing 8: LLM deployment to Android device

6 Voice Interface

6.1 Speech-to-Text (Vosk)

```
fun initializeVosk() {
    val modelDir = File(context.cacheDir, "model-es-small")
    if (!modelDir.exists()) {
        unpackAssets("model-es-small", modelDir)
    }
    voskModel = Model(modelDir.absolutePath)
}

fun startListening() {
    val recognizer = Recognizer(voskModel, 16000.0f)
    speechService = SpeechService(recognizer, 16000.0f)
    speechService?.startListening(object : RecognitionListener {
        override fun onResult(hypothesis: String?) {
            hypothesis?.let {
                val json = JSONObject(it)
                val text = json.getString("text")
                if (text.isNotBlank()) onResult?.invoke(text)
            }
        }
    })
}
```

Listing 9: Vosk STT integration

7 Results and Evaluation

7.1 Performance Metrics

Table **??** shows the performance metrics for each component.

Table 2: Performance metrics on mid-range Android device (Snapdragon 6xx)

Component	Latency	Memory	Accuracy
Vision Model	200-400ms	50MB	92.3% Top-1
Semantic Search	60-130ms	100MB	94.4% Top-1
LLM Generation	2-5s	1.2GB	–
Voice STT	Real-time	100MB	85-92%

7.2 Disease Detection Results

The vision model achieves:

- **Top-1 Accuracy**: 92.3%

- **Top-3 Accuracy**: 98.1%
- **Inference Time**: 200-400ms

8 Deployment Workflow

Figure **??** illustrates the complete deployment workflow from model training to device installation.

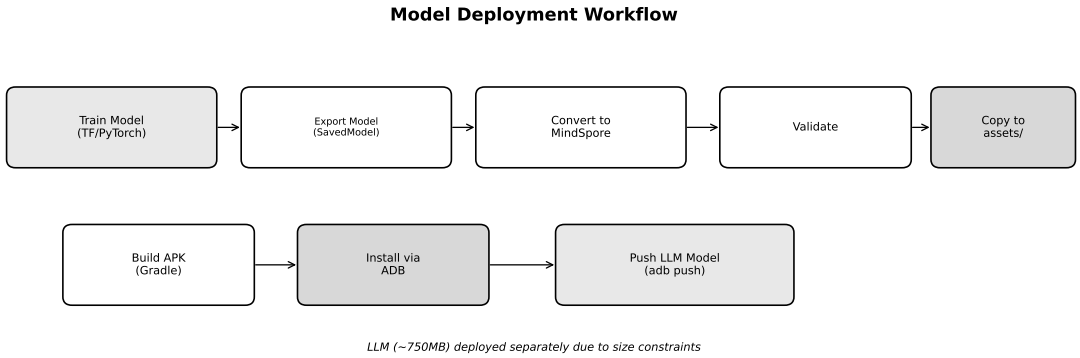

Figure 6: Model deployment workflow

9 Datasets and Data Sources

9.1 Vision Model Training Data

The plant disease classification model was trained using two complementary datasets:

PlantVillage Dataset: A publicly available dataset containing 54,309 images of healthy and diseased plant leaves across 14 crop species. We extracted images for our target crops (tomato, potato, pepper, corn) covering 18 disease classes plus healthy states. Available at: `https://github.com/spMohanty/PlantVillage-Dataset` and `https://www.kaggle.com/datasets/emmarex/plantdisease`.

BRACOL Coffee Dataset: The Brazilian Arabica Coffee Leaf dataset (BRACOL) containing 1,747 images of arabica coffee leaves affected by the main biotic stresses. The dataset includes four disease classes:

- Leaf miner (Minador)
- Leaf rust (Roya)
- Brown leaf spot
- Cercospora leaf spot

Available at: `https://data.mendeley.com/datasets/yy2k5y8mxg/1` (Paper: `https://doi.org/10.1016/j.compag.2019.105162`).

Both datasets were preprocessed to 224×224 pixels and augmented using rotations, flips, zooms, and brightness variations to increase model robustness.

9.2 NLP Knowledge Base

The semantic search component uses a curated agricultural knowledge base consisting of:

- 517 question-answer pairs covering crop management, disease identification, pest control, and best practices
- Questions sourced from agricultural extension services, farmer forums, and expert consultations
- Content validated by agronomists from the University of Cauca
- Categories: Crop Management (35%), Disease Control (30%), Pest Management (20%), Soil & Fertilization (15%)

Each entry includes multiple question variations to improve semantic matching, with embeddings pre-computed using the paraphrase-multilingual-MiniLM-L12-v2 model.

9.3 Voice Recognition Model

The offline speech recognition uses Vosk's pre-trained Spanish language model (`vosk-model-small-es-0.42`):

- Training data: Common Voice Spanish corpus + proprietary datasets
- Model size: 50MB compressed
- Optimized for Latin American Spanish dialects
- Adapted vocabulary includes agricultural terminology

Available at: `https://alphacephei.com/vosk/models/vosk-model-small-es-0.42.zip` (Official models: `https://alphacephei.com/vosk/models`).

9.4 Language Model

The Llama 3.2 1B Instruct model was developed by Meta AI:

- Pre-trained on diverse internet text and code
- Fine-tuned for instruction-following tasks
- Quantized to Q4_K_M format using llama.cpp
- No additional domain-specific fine-tuning applied

Available at: `https://huggingface.co/meta-llama/Llama-3.2-1B-Instruct` (Quantized: `https://huggingface.co/bartowski/Llama-3.2-1B-Instruct-GGUF`).